

Here is a **clean, structured, evaluator-ready manual testing document for Minishell** based strictly on the sheet you provided. It includes:

- Exact **commands to run**
- What the **expected behavior/output** is
- Clear sections following the **full mandatory evaluation sheet**
- No bonus
- No filler, only what an evaluator needs

Minishell – Manual Evaluation Testing Guide (Mandatory Only)

A practical step-by-step checklist with commands and expected behavior.

1. Compilation Tests

1.1 Check Makefile Flags

```
make -n
```

Expected: Compilation commands must include:

```
-Wall -Wextra -Werror
```

1.2 Build the Program

```
make
```

Expected: No compilation error.

1.3 Re-run Make

```
make
```

Expected:

```
Nothing to be done
```

(no re-linking)

2. Basic Execution & Simple Commands

Launch minishell:

```
./minishell
```

2.1 Absolute Path

```
/bin/ls
```

Expected: Output identical to bash's `/bin/ls`.

2.2 Empty Input

```
<press ENTER>
```

Expected: New prompt, nothing executed.

2.3 Only Spaces

```
<space><space><ENTER>
```

Expected: Same as empty input.

3. Arguments & History

Normal Arguments

```
/bin/ls -l  
/bin/echo hello world  
/bin/cat Makefile
```

Expected: Same behavior as bash.

Repeat multiple times. **History:** Press ↑ and ↓ to navigate previous commands.

4. echo

Basic

```
echo hello
```

Expected:

```
hello
```

-n Flag

```
echo -n hello
```

Expected:

```
hello<no newline>
```

Empty Arguments

```
echo
```

Expected: Prints newline.

5. exit

Basic

```
exit
```

Expected: Minishell closes.

Numeric Argument

```
exit 42
```

Expected: Minishell exits with code 42 (check with `/bin/echo $?` after running in bash).

Non-numeric Argument

```
exit hello
```

Expected: Bash behavior:

```
exit: hello: numeric argument required
```

Minishell must mimic this and exit with code 255.

6. Return Value (\$?)

Success

```
/bin/ls  
echo $?
```

Expected:

```
0
```

Failing Command

```
/bin/ls does_not_exist  
echo $?
```

Expected: Non-zero (bash prints 2 for file not found).

Math Example

```
/bin/false  
echo $?  
expr $? + $?
```

Expected:

- `/bin/false` → exit code 1
 - `expr 1 + 1` → prints 2
-

7. Signals

Empty Prompt

- **Ctrl-C** → new line + new prompt
- ***Ctrl-*** → nothing
- **Ctrl-D** → exit minishell

Typing but not executing

Type `hello` then press:

- **Ctrl-C** → clears line + new prompt
- **Ctrl-D** → does nothing
- ***Ctrl-*** → exits minishell

Blocking Command

Run:

```
cat
```

Press signals:

- **Ctrl-C** → should interrupt and return to prompt
- ***Ctrl-*** → should quit cat and return to prompt
- **Ctrl-D** → should send EOF (ends `cat`) and return to prompt

8. Double Quotes

```
echo "hello world"
```

Expected:

```
hello world
```

```
echo "cat lol.c | cat > lol.c"
```

Expected: Prints literally:

```
cat lol.c | cat > lol.c
```

9. Single Quotes

```
echo '$USER'
```

Expected:

```
$USER
```

```
echo ' spaced text '
```

Expected: Prints exactly the content.

Nothing inside single quotes is interpreted.

10. env

```
env
```

Expected: List of all environment variables, same as `bash`.

11. export

Add New Variable

```
export TEST=hello
env | grep TEST
```

Expected:

```
TEST=hello
```

Replace Variable

```
export TEST=world  
env | grep TEST
```

Expected:

```
TEST=world
```

12. unset

```
export TO_REMOVE=42  
unset TO_REMOVE  
env | grep TO_REMOVE
```

Expected: Nothing printed.

13. cd

To Home

```
cd  
pwd
```

Expected: Home directory.

Relative

```
cd ..  
pwd
```

Invalid Directory

```
cd does_not_exist
```

Expected: Error message similar to bash.

Special

```
cd .  
cd ..
```

14. pwd

```
pwd
```

Expected: Correct current directory.

Test in multiple directories.

15. Relative Executables

Go to minishell directory:

```
cd src
../minishell
```

Inside minishell:

```
../minishell
```

Expected: Should execute the program using a relative path.

16. PATH Resolution

Commands without path

```
ls
wc minishell.c
```

Expected: Works if PATH is valid.

Unset PATH

```
unset PATH
ls
```

Expected:

```
ls: command not found
```

Custom PATH

```
export PATH=/usr/local/bin:/bin
ls
```

Expected: Should search left-to-right and execute once found.

17. Redirections

Output Redirection

```
echo hello > out.txt
cat out.txt
```

Expected:

```
hello
```

Append

```
echo world >> out.txt
cat out.txt
```

Expected:

```
hello
world
```

Input Redirection

```
cat < out.txt
```

Expected: Same content printed.

Heredoc

```
cat << END
hello
END
```

Expected: Prints:

```
hello
```

Invalid Multiple Redirections

```
echo hello >>> file
```

Expected: Error similar to bash.

18. Pipes

```
cat out.txt | grep hello
```

Expected:

```
hello
```

```
ls does_not_exist | grep hi
```

Expected: `ls` prints error, pipe continues.

Pipes + Redirections

```
cat out.txt | grep hello > result.txt
cat result.txt
```

Expected:

```
hello
```

19. Stress Tests & History

Interrupt Line

Type:

```
echo something
```

Press **Ctrl-C**, then **ENTER** **Expected:** Nothing executed.

History Navigation

Press **↑** multiple times. **Expected:** Should scroll through all previous commands.

Invalid Command

```
asuydgfagsyu
```

Expected:

```
asuydgfagsyu: command not found
```

Shell must not crash.

Long Command

```
echo aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

Expected: Prints the long string.

20. Environment Variables Expansion

Basic

```
echo $USER
```

Expected: Prints correct username.

Inside Double Quotes

```
echo "$USER"
```

Expected: Same as above.

Multiple Variables

```
echo $USER $HOME $PATH
```

Undefined Variable

```
echo $NOTSET
```

Expected: Prints empty string.

End of Evaluation

If all sections work without crashes and without leaks → **Pass**.

If you want, I can also create a **PDF checklist**, or a **print-ready evaluation sheet** you can use during the defense.